



Recovery Without Reintroducing Risk

A Security-First Approach to RTO/RPO in Event-Driven Systems

By Sanjay Iyer Kudali Prasanna Kumar
Independent Researcher

Conf42 DevSecOps 2026

The Core Problem

Your System Came Back Online. But Did It Come Back **Safe?**

RTO and RPO confirm uptime is restored. They say nothing about whether the system came back **correct**, or whether the recovery itself opened a new attack surface.

The Hidden Failure Mode in Recovery

A recovery that meets every resilience benchmark can still quietly reintroduce security and compliance failures: ones that conventional metrics will never surface.



Damage Dimensions in Event-Driven Recovery

Five Ways a "Successful" Recovery Can Fail

M Missing Events

A window closed on incomplete input, or a replay that started past the true frontier. The aggregate is published, and it is wrong.

U Duplicate Effects

Events replayed past the deduplication horizon. A charge, a notification, or a fraud decision executes twice.

O Ordering Inversions

An older update applied after a newer one. Business logic and downstream state diverge from the source of truth.

S State Staleness

Restored state older than what was already committed, including authorization and session state that grants access it should no longer hold.

N Non-compensable Divergence

An external effect that cannot be undone: a payment settled, a message sent, a record disclosed. The one dimension you cannot trade away.



Together these form the damage vector $D = (M, U, O, S, N)$, each bounded over an explicit key scope and time horizon.



Why This Matters

Passing Resilience Benchmarks $\neq$ Restoring Trust

Teams can satisfy every SLA target while silently reintroducing exploitable state during the recovery process itself. The gap between **uptime** and **integrity** is exactly where security and compliance failures hide.

⚠ Conventional recovery metrics are blind to this class of risk.

Introducing the Framework

The Semantic Recovery Envelope

A decision model that makes recovery correctness explicit **before a plan executes**, and that allows a controller to reject the fastest recovery plan when that plan produces greater logical harm.

Bound the Damage

Map collected evidence to a conservative upper bound on each of the five dimensions, for every candidate plan. Unknown evidence widens the bound; it is never treated as safe.

Filter by Hard Constraints

Remove every candidate whose bound violates a hard limit, such as zero non-compensable divergence for irreversible actions. Safety first, optimization second.

Select Among Survivors

Pareto-rank what remains, apply policy weights last, and journal the decision. The plan may be replay, compensation, isolation, read-only mode, or a delayed promotion.

The Evidence Model

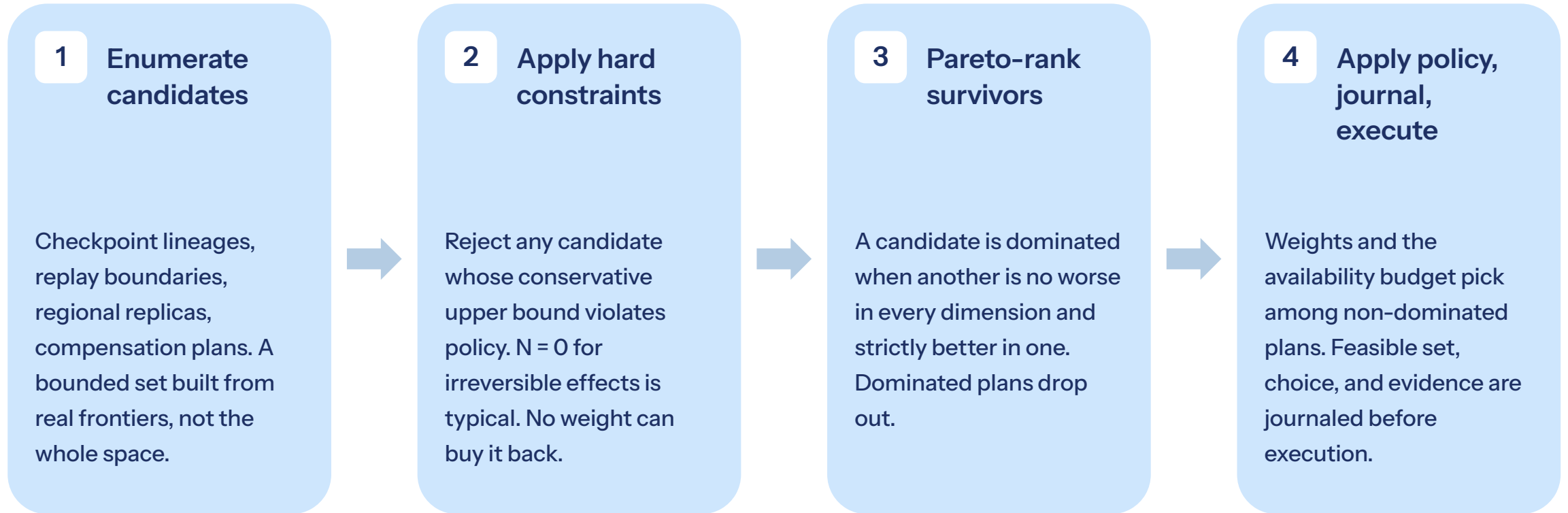
Evidence Is Collected, Never Inferred

A recovery candidate is defined by its checkpoint lineage, replay interval, state schema, sink-applied frontier, deduplication horizon, and outstanding external effects. Each is proved by a specific attestation, and each attestation bounds specific dimensions.

Evidence	What it proves	Dimensions it bounds
Source offsets	Which input the candidate actually covers	M
Event-time frontiers (watermarks)	Whether windows and joins closed on complete input	M, O
State digests	Which state version was restored, not which one you assume	S, O
Sink receipts	Which outputs are committed downstream	U, S
External-effect ledger	Which actions ran outside the transactional boundary	U, N

 Fail closed. A missing sink receipt or effect acknowledgement cannot be turned into proof by a better optimizer. Unknown evidence expands the envelope and can force isolation, compensation, or replay into a separate namespace.

Hard Constraints, Then Pareto, Then Policy



 The fastest plan is rejected when it produces more logical harm. That is the point of the model, not a side effect. Safety is decided before optimization, so the safety decision can be audited on its own.

Mapping Damage Dimensions to Responses

Dimension	Security reading	Response that bounds it
M Missing events	Incomplete aggregates published as complete; gaps in the audit record	Replay from an older checkpoint; delay window close or promotion until the frontier catches up
U Duplicate effects	Double charges, repeated notifications, fraud controls bypassed by replay	Replay only inside the deduplication horizon; compensate effects already applied beyond it
O Ordering inversions	Older updates overwrite newer ones; state diverges from the source of truth	Replay in source order from a consistent checkpoint; fence writers from older epochs
S State staleness	Revoked permissions and expired sessions restored as valid	Reject candidates older than sink-committed state; isolate keys until fresh; hard bound for authorization state
N Non-compensable divergence	Irreversible actions repeated or lost; a compliance boundary crossed	Hard constraint $N = 0$: stop irreversible actions, quarantine affected keys, never replay across the effect boundary

 Dimensions compound. One failure event moves several at once; the envelope evaluates all five before a response is chosen.

Five Responses, Each With a Security Posture

Replay

Re-execute input from a checkpoint. Bounds M and O; raises U unless replay stays inside the deduplication horizon or effects are idempotent.

Compensation

Emit corrective events for effects already applied. The only way to cut U past the effect boundary; impossible for non-compensable effects.

Traffic Isolation

Quarantine affected keys or a region and fence older epochs. Stops stale or duplicate state from reaching healthy consumers.

Rollback

Restore an older checkpoint lineage. Clears a suspect epoch's duplicates and inversions; raises S, and is unsafe if newer state was already sink-committed.

Partial Service

Read-only or last-known-good reads for uncertain keys. Holds S and N flat while evidence catches up. Costs availability, not integrity.

Recovery Epochs and Four Machine-Checkable Properties

Each plan receives a monotonically increasing epoch, and workers reject writes from older epochs. Decisions are made per key range when the evidence supports independent recovery. All evidence is read at one stable decision frontier; anything arriving later belongs to the next planning round.

1 Safety

A plan activates only when the conservative upper bound for every hard dimension satisfies policy.

2 Epoch Exclusivity

At most one recovery epoch may commit output for a key range. This is the split-brain guard on failover.

3 Evidence Coherence

Every source, state, sink, and effect attestation used by a decision is bound to one immutable decision frontier and schema version.

4 Decision Reproducibility

Given the journaled evidence, policy version, and planner version, an auditor obtains the same feasible set and the same selected plan.

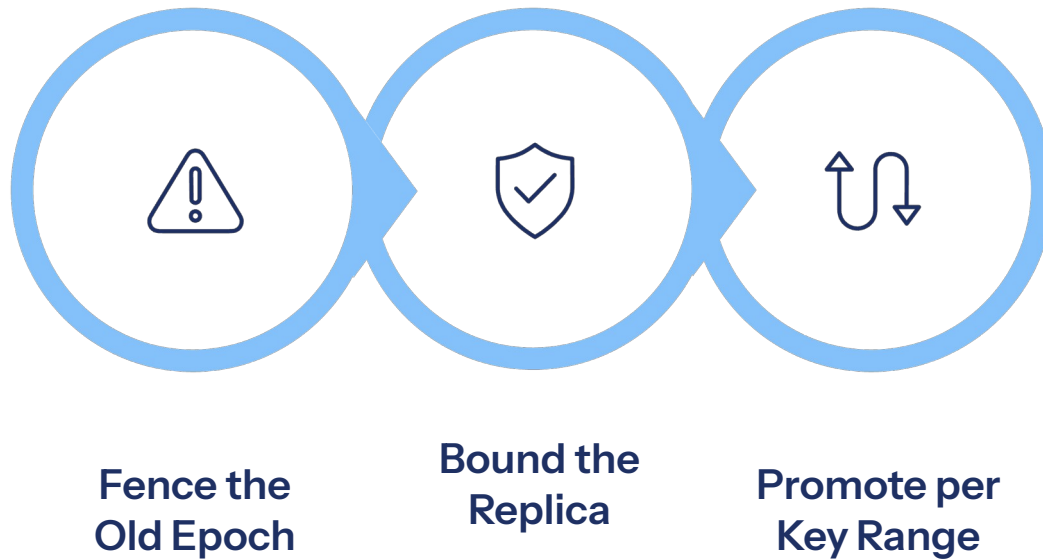


How These Decisions Hold Up Under Real Operating Conditions

Three scenarios every team actually faces: a regional failover, a lost sink receipt, and a split-state recovery where some consumers are ahead and some are behind.

Scenario 1

Regional Failover



Promoting a standby without bounding its state is a common source of permission regressions in failover. The Envelope fences the old region's writers first, reads the replica's sink-applied frontier and state digests at one decision frontier, and promotes only the key ranges whose bounds are proven.

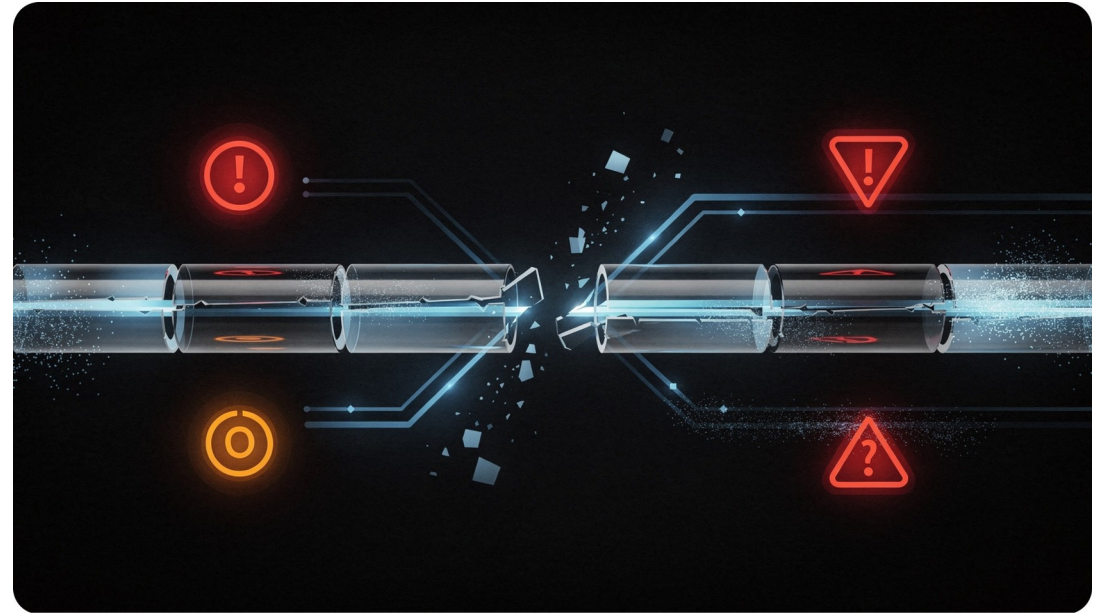
- **Dimensions in play:** S, O, M
- **Evidence:** replica frontier versus last committed frontier, state digests, effect ledger
- **Outcome:** promoted ranges plus explicitly isolated ranges, never a blind global switch

Scenario 2

The Lost Sink Receipt

The Risk

The consumer offset says the event was processed. The receipt that proves the sink write is missing. Treating the offset as proof of completion is how a replay double-charges a customer, and how a non-compensable effect gets repeated.



Detect

The missing receipt widens the U bound for the affected keys. If the effect class is irreversible, $N > 0$ becomes possible and the replay candidate is rejected.

Compensate

Consult the effect ledger. Compensate applied effects that are compensable, and emit tombstones before any forward replay.

Verify

Replay only inside the deduplication horizon, per key range, and confirm sink receipts before restoring full service.

Scenario 3

Split-State Recovery

State of the key range	Prescribed response
Evidence complete, all bounds within policy	Resume: execute the selected plan for that range, replay or promote.
Evidence complete, a soft bound exceeded	Trade explicitly: compensate, roll back, or delay. Pareto ranking and policy weights decide, and the choice is journaled.
Evidence complete, a hard bound violated ($N > 0$)	Reject the candidate: stop irreversible actions, quarantine the range, serve last-known-good reads.
Evidence unknown	Fail closed: the bound expands; the range stays isolated or read-only until evidence arrives. Break-glass only with a named damage bound.

When only part of the infrastructure recovers, some consumers are ahead and some are behind. Deciding per key range keeps one uncertain shard from blocking the whole service, and keeps "just replay everything" from amplifying the incident.

No judgment calls under pressure: each state has a prescribed response, and the response is recorded.

What to Instrument Before You Need It

Signals the planner must publish

Per-range evidence-age histograms

How stale is the proof you are deciding on? A decision on stale evidence is a stale decision.

Rejected-candidate counts by constraint

Which hard limit is actually doing the work, and how often the fastest plan lost.

Fraction of ranges with unknown bounds

Your fail-closed exposure. The number on-call will watch most closely.

Planner latency and journal durability lag

The decision must be durable before it executes. Lag here is a correctness risk, not a performance one.

Cost envelope

Dominance pass is $O(R \cdot C^2)$

R independently recoverable ranges, C candidate frontiers per range, five fixed dimensions. Lineage pruning and monotonic frontiers keep C small.

Control path stays off the data path

Only recovery evidence and effect identifiers are retained, so storage scales with recovery granularity, not with event volume.

Exact ledgers only where they must be exact

Mandatory for non-compensable effects. Tolerable dimensions may use sketches, as long as each sketch carries an explicit confidence bound.

Fail Closed, Break Glass Deliberately, Roll Out Policy Like Code

Planner outage

Existing epochs and ownership stay valid. New promotions fail closed. An outage of the decision layer never becomes an unverified promotion.

Break-glass

A separately audited policy an operator may invoke only when it names the accepted damage bound. Emergency access that cannot state its blast radius is not break-glass. It is bypass.

Policy rollout

Validate the new policy against retained traces, shadow both versions, then atomically advance the policy epoch. Roll back only if no action authorized solely by the new policy crossed a non-compensable boundary.



Insufficient evidence is never permission to resume.

Beyond RTO and RPO

Recovery Objectives That Include Correctness

RTO and RPO stay. The envelope adds five bounded objectives, set per workload and per key scope, with hard limits where the damage cannot be undone.

Continuity objectives

RTO

How fast can the system accept traffic again?

RPO

How much data loss is tolerable?

Envelope objectives, per key scope and time horizon

M Missing-event exposure

How many events, or how much of a window, may be absent from a published result?

U Duplicate-effect exposure

How many replayed effects may reach a sink or a customer?

O Ordering-inversion exposure

How many older updates may land after newer ones?

S State-staleness exposure

What is the maximum age of restored state? Authorization state is bounded separately, and more tightly.

N Non-compensable divergence

Zero for irreversible actions. Not a weight; a prohibition.

Validation & Audit Confidence

Building Evidence That Holds Up to Security Review



What Auditors and Security Reviewers Actually Ask For

- **Pre-execution attestation:** the evidence used, bound to one decision frontier and schema version, journaled before the plan runs
- **Strategy rationale:** the feasible set, every rejected candidate, and the constraint that rejected it
- **Post-recovery validation:** a verifier confirms the selected envelope was not exceeded after execution
- **Reproducibility:** the same journaled evidence, policy version, and planner version give an auditor the same plan

Key Takeaways

What You Leave With

1 Recovery Is a Semantic Planning Problem

Five bounded damage dimensions, measured per key scope and horizon, supplement two scalars that hide the tradeoffs that matter.

2 Safety Before Optimization

Hard constraints, then Pareto, then policy. The fastest plan can lose, and when it does, the reason is written down.

3 Evidence, Not Inference

Unknown evidence widens the bound. Fence epochs, journal the decision, and let an auditor reproduce it.



Restoring uptime is necessary. Restoring integrity is non-negotiable.

Further Reading

References

[1] P. Carbone, G. Fóra, S. Ewen, S. Haridi, K. Tzoumas. Lightweight Asynchronous Snapshots for Distributed Dataflows. arXiv:1506.08603, 2015.

[2] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, K. Tzoumas. State Management in Apache Flink: Consistent Stateful Distributed Stream Processing. PVLDB 10(12), 2017.

[3] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, R. Rabiser. A Comprehensive Benchmarking Analysis of Fault Recovery in Stream Processing Frameworks. DEBS 2024.

[4] K. Mamouras, C. Stanford, R. Alur, Z. G. Ives, V. Tannen. Data-Trace Types for Distributed Stream Processing Systems. PLDI 2019.

[5] T. Akidau et al. Watermarks in Stream Processing Systems: Semantics and Comparative Analysis of Apache Flink and Google Cloud Dataflow. PVLDB 14(12), 2021.

[6] Apache Kafka documentation: message ordering, consumer position, and exactly-once semantics. kafka.apache.org/documentation

[7] Z. Cheng, Q. Huang, P. P. C. Lee. On the Performance and Convergence of Distributed Stream Processing via Approximate Fault Tolerance. arXiv:1811.04570.

[8] M. K. Geldenhuys et al. Khaos: Dynamically Optimizing Checkpointing for Dependable Distributed Stream Processing. arXiv:2109.02340, 2021.

Thank You!